

Activity 1: MQTT communication with ESP32 and server

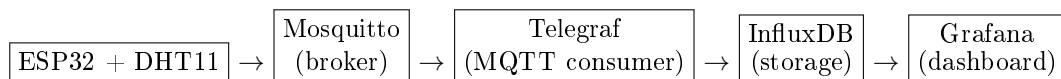
Omar Antonio Cornejo Vargas

1 October 2026

1 Introduction

The objective of this activity is to build a complete IoT telemetry chain for a DHT11 sensor connected to an ESP32: the board reads the temperature and the relative humidity, publishes them over MQTT, a server-side pipeline stores the measurements in a time-series database, and a dashboard displays them graphically.

The architecture used is a publish/subscribe pipeline in which every component runs in a Docker container, so the whole system is reproducible and isolated from the host:



Mosquitto was selected as the broker, Telegraf as the bridge between MQTT and the database, InfluxDB 1.8 as the storage engine (InfluxQL) and Grafana for the visualization. This report describes the work done, the key decisions taken and the problems found, presents the results obtained, answers the questions of the activity and ends with the conclusions.

2 Work Done

2.1 DHT sensor communication

The firmware (`sketch_sep22b/allcode.ino`) uses the `DHT.h` library to read the DHT11 sensor connected to pin 33. Values are validated with `isnan()` before anything is sent. **Decision:** the sensor is polled in the main `loop()` with a 1 s delay, keeping the readings meaningful without flooding the broker.

2.2 MQTT sending

The board connects to the broker with `WiFiClient` and `PubSubClient`, and reconnects automatically if the connection is lost (checked on every loop iteration).

Decision: publish a single JSON payload to a single topic instead of one topic per variable. The first version of the firmware published two topics (`sensor/temperature` and `sensor/humidity`) with plain numeric values. This was changed to a single topic `sensor/esp01` carrying a JSON object with the two measurements. **Reason:** with one topic and one JSON object the broker carries a single message per reading, Telegraf creates a single measurement with two fields.

2.3 Data treatment with Telegraf (MQTT consumer)

Telegraf is the bridge between the broker and the database. It runs the `inputs.mqtt_consumer` plugin, which opens a persistent MQTT connection to Mosquitto and subscribes to `sensor/#`, the wildcard that matches any topic under `sensor/`. With that subscription, every message published by the ESP32 is received without having to configure anything new in the pipeline, and if a second sensor were added publishing on `sensor/esp02` it would also be ingested.

The `data_format = "json"` option tells Telegraf that the payload is a JSON object instead of a bare value, so each key of the object is turned into a field of the metric: `temperature` and `humidity`. The option `name_override = "mqtt_consumer"` fixes the name of the metric, so the measurements do not depend on the topic name and InfluxDB always receives the same measurement. Therefore one message received from the topic `sensor/esp01` is converted into a metric that contains two fields and one tag, `host`, which identifies the container that produced the data:

```
{"temperature":24.10,"humidity":76.00} -> fields: temperature, humidity tag: host
```

Problem found: the initial configuration used `data_format = "value"`, which only accepts a bare number, so nothing was written when the JSON payload was published. **Solution:** switch to `"json"` and remove the per-topic filters.

2.4 Data treatment with InfluxDB (storage)

The `outputs.influxdb` plugin sends every metric to InfluxDB over HTTP. In that protocol the name of the measurement comes first, the tags follow after it, and then the fields with their values, so the message shown above is stored as:

```
mqtt_consumer,host=8fd0addeff46 temperature=24.1,humidity=76 1790785562000000000
```

The temperature and the humidity are stored as two float fields of the same measurement and share the timestamp of the MQTT message, which means that every reading is kept as one point and can be queried directly, for example with `SELECT last("temperature"), last("humidity") FROM "mqtt_consumer"`.

2.5 Visualization (Grafana)

Grafana is provisioned with an InfluxDB datasource and a file-provisioned dashboard with two time-series panels, one for temperature (°C) and one for humidity (%), both aggregating with `mean()` over the dashboard interval.

2.6 Deployment and code

Everything described above is packaged in a single `docker-compose.yml`: the broker, the database, the consumer and the dashboard are defined together, so the whole server side is started with one command, `docker compose up -d`, and stopped with `docker compose down`. The code is available at <https://github.com/omar-cornejo/EPSEM>.

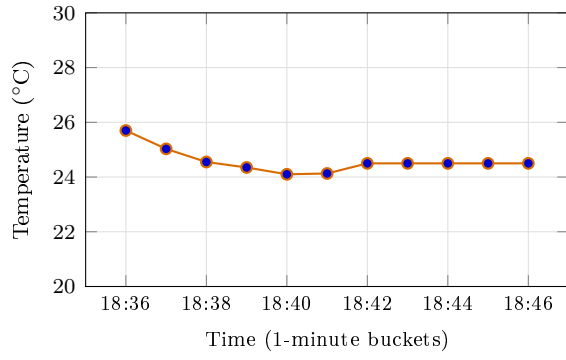
3 Results

The measurements flow end to end: in the last ten minutes of the session InfluxDB stored 580 points in the measurement `mqtt_consumer`, with a mean temperature of 24.44 °C and a mean humidity of 78.9%.

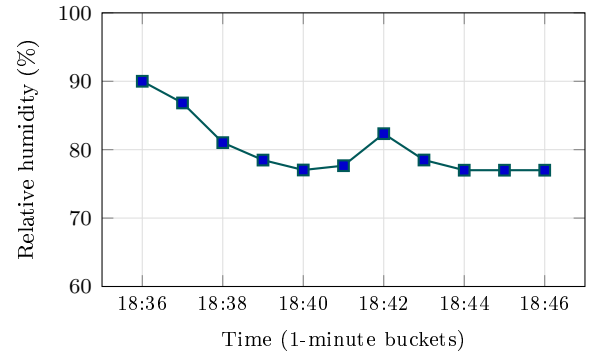
Figures 1a and 1b show the last ten minutes of that stream, averaged in one-minute buckets and drawn side by side. The values were exported directly from InfluxDB with the `influx` CLI and saved in `data/sensor.csv`.

In the exported data the temperature stays between 24.1 and 25.7 °C and the humidity between 77% and 90%, both values dropping after a short peak and then staying flat, which is consistent with the two fields being measured in the same reading.

The Grafana dashboard shows the same data over a longer window, one hour, which makes it possible to check whether the values in the room change abruptly. Figure 2 shows the two panels of that dashboard. When the temperature or the humidity of the environment changes drastically



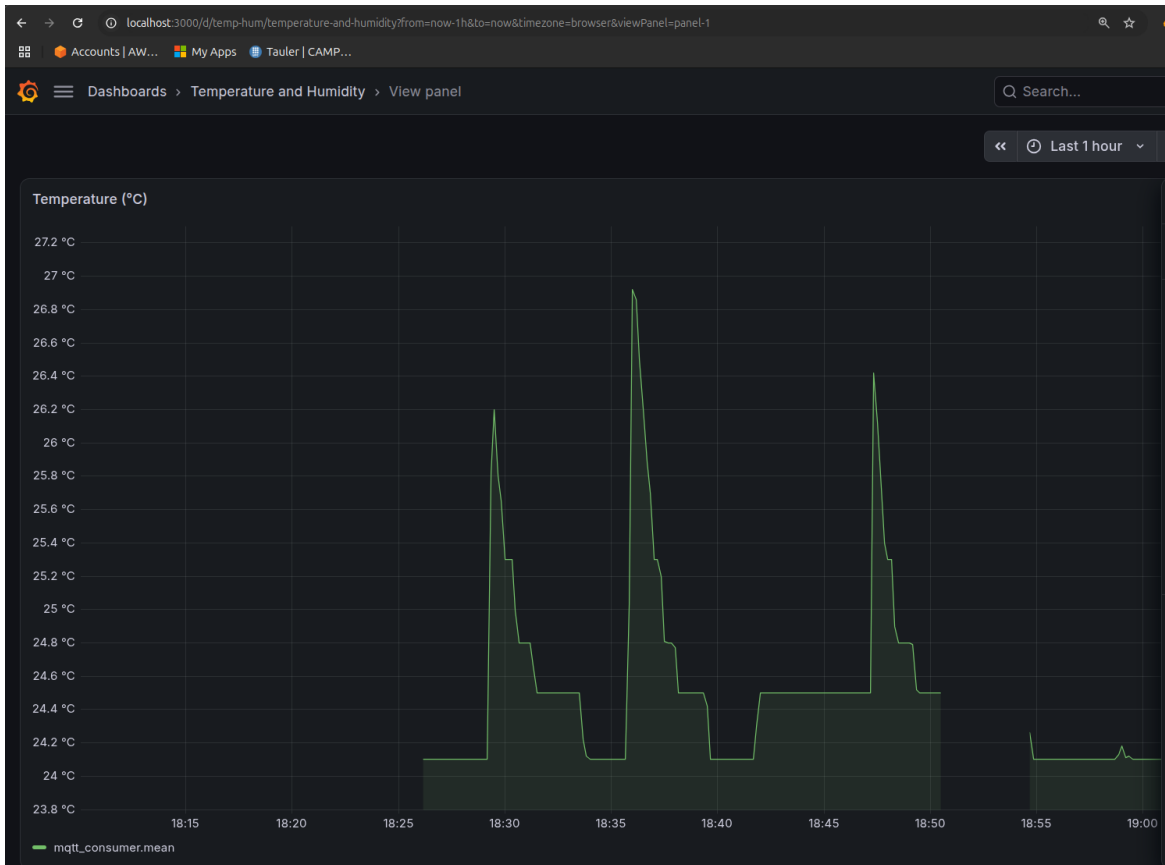
(a) Temperature stored in InfluxDB, stable around 25 °C indoors.



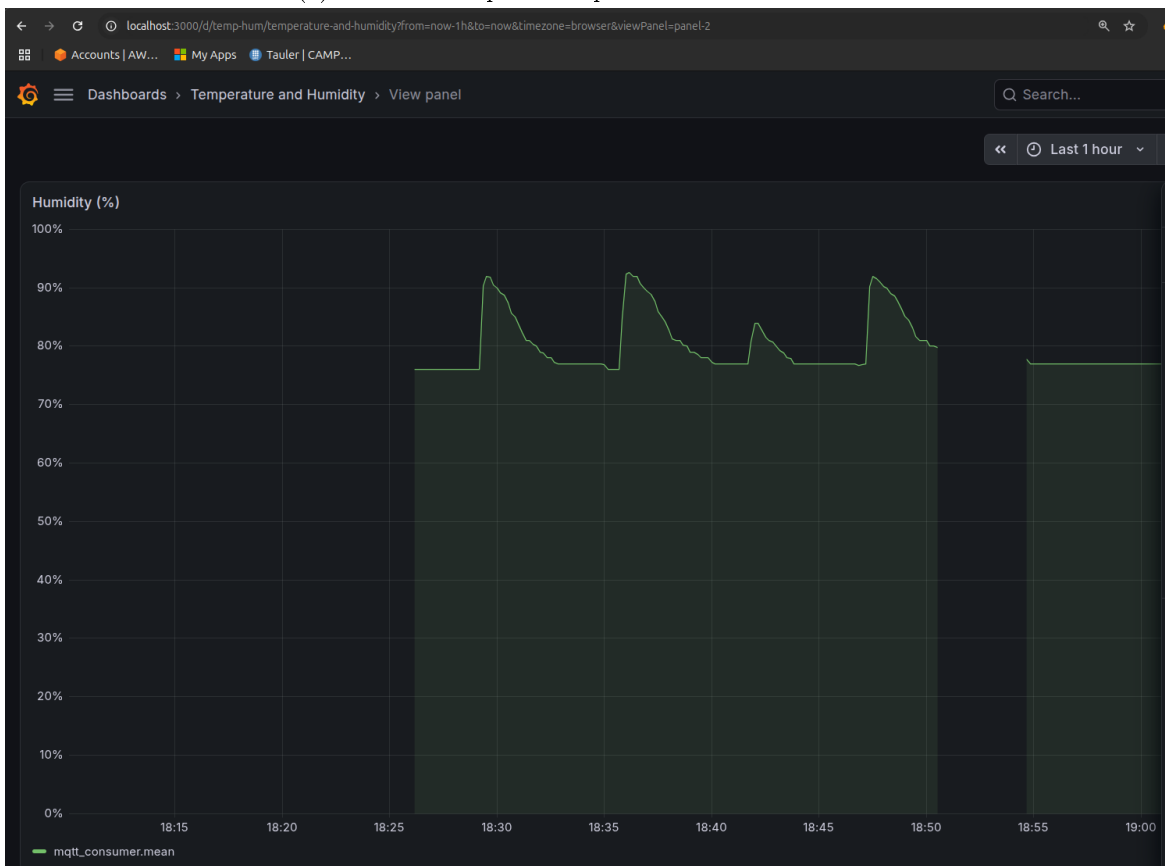
(b) Relative humidity, following the same shape with a wider spread.

Figure 1: Last ten minutes of the stream stored in InfluxDB.

the corresponding graph spikes. Normally both graphs stay flat and stable, which is the expected behaviour of a DHT11 placed in a room.



(a) Grafana temperature panel over the last hour.



(b) Grafana humidity panel over the last hour.

Figure 2: The two panels of the Grafana dashboard over the last hour of data.

4 Answers to the Questions

Why is MQTT suitable for IoT? MQTT is designed for constrained devices: header is only a few bytes and there is no HTTP overhead, so a reading costs very little bandwidth and power, which matters for battery-powered nodes.

How does pub/sub improve efficiency? The publisher does not know or address subscribers: it simply publishes to a topic, and the broker delivers the message to whoever is subscribed. The sensor therefore publishes only when it has a new reading, instead of polling a server or opening a connection per reading, and the same message can feed several consumers (storage, dashboard, alerts) without duplicating traffic. Communication is asynchronous, so neither side has to wait for the other.

Advantages of running Mosquitto in Docker. The installation is reproducible from a single `docker-compose.yml`, the broker is isolated from the rest of the host system, configuration and data live in bind-mounted volumes that survive container restarts, updating or testing another broker version is a matter of changing the image tag, and the same file lets any other member of the course run the exact same environment.

5 Conclusions

The activity has been completed: the ESP32 reads the DHT11, publishes a single JSON payload with the temperature and the humidity to `sensor/esp01`, and the Docker pipeline (Telegraf, InfluxDB, Grafana) stores and visualizes the data, with the results verified both from Grafana and from the InfluxDB command line.